

Dissecting GPU Utilization for LLM Inference on Nvidia Hopper

Mohammad Siavashi, Gerald Q. Maguire Jr., Dejan Kostić, and Marco Chiesa
 KTH Royal Institute of Technology, Stockholm, Sweden
 {siavashi, maguire, dm, mchiesa}@kth.se

Abstract

A single “SM utilization” percentage can make an LLM inference workload look compute-saturated while hiding how much useful work is being done. The problem is not that the counter is wrong, but that it collapses several different mechanisms into one number. This is most severe during decode, where each request contributes only one new token and dense projection GEMMs become small-row matrix multiplications. On Hopper, the bfloat16 GMMMA path executes these operations in fixed 64-row matrix fragments, so small-batch decode can fill only a small fraction of each fragment with real token rows.

In this paper, we profile vLLM with FlashAttention-3 and cuBLASLt on an H100 NVL across cold prefill, warm prefill, and decode, sweeping sequence length and batch size. We replace the usual single utilization number with eight counter-validated views derived from raw Nsight Compute reports, each pinned to an NCU counter or explicit formula. Together, these views map utilization gaps to concrete mechanisms—fragment fill, occupancy limits, stall signatures, wave quantization, and kernel selection—across four production models and six per-layer kernel roles.

Index Terms—LLM inference, GPU workload characterization, NVIDIA H100, Hopper, Tensor Cores, Nsight Compute, FlashAttention-3, mixture of experts, vLLM, performance profiling.

1. Introduction

A GPU can look busy for reasons that have little to do with useful LLM work. That distinction now matters in practice: LLM inference consumes enough hyperscale GPU time that a utilization number quickly becomes a capacity-planning, batching, and optimization decision. Yet the question “is the GPU being used?” is often answered with a single percentage: NVIDIA Nsight Compute (NCU)’s `sm__throughput.avg.pct_of_peak_sustained_elapsed`, conventionally called *SM utilization* or `sm_busy_pct`. This counter is useful, but narrower than the name suggests. It is an elapsed-cycle pipeline-throughput proxy, not the `sm__cycles_active` “ ≥ 1 warp resident” counter, not an occupancy measurement, and not a direct measure of useful matrix-multiply throughput. The problem is not that the counter is wrong; it is that a single scalar is being asked to explain too many different denominators.

This ambiguity is concrete. In Qwen3-14B dense decode on an H100 at batch size 32, the GEMMs report $\approx 72\%$ `sm_busy_pct`, reading as near compute-saturation—yet only half of each Hopper m64 matrix fragment holds real token

rows, and `long_scoreboard` memory stalls still dominate the issue budget; the cause is architectural, an m64 fragment floor in Hopper’s bfloat16 (BF16) warp-group matrix multiply-accumulate (GMMMA) path that no tile retuning can repair (§5.6). The same ambiguity appears the moment the serving regime changes. For the same projection role at the same model, context length S , and batch size B , `sm_busy_pct` can be $\sim 97\%$ in cold prefill, where cuBLASLt’s autotuner selects a large tile, yet only $\sim 9\%$ in warm prefill after a cached prefix collapses the row dimension toward decode size (§5.6). Even occupancy can appear to tell two stories at once: the same kernel is only $\sim 14\%$ occupied against the architectural 64-warp ceiling, but 75–95% occupied against its own register/shared-memory resource cap (§2.4). Each reading is internally consistent. The confusion comes from forgetting which denominator each metric uses.

This paper makes those denominators explicit. We instrument vLLM with FlashAttention-3 (FA3), cuBLASLt, and Hopper’s GMMMA path on an H100 NVL with 94 GB third-generation high-bandwidth memory (HBM3) [15]. We study four production models: Meta-Llama-3-8B, Qwen3-14B, Qwen3-32B, and Qwen3-30B-A3B, the last a sparse mixture of experts (MoE) model with top-8 routing over 128 experts. For each model, we sweep three serving regimes—cold prefill, warm prefill, and decode—along with context length S , batch size B , and per-layer kernel role: QKV projection, attention output projection, FFN Gate/Up projection, FFN Down projection, attention, and the MoE expert path where applicable. For every (model, regime, S , B , role) point, we compute eight utilization views directly from raw Nsight Compute report files, pinning each view either to an NCU counter or to an explicit formula.

The central result is that decode underutilization is not one bottleneck but several—resource limits, spatial wave quantization, and an instruction-set fragment floor—each with a different fix, and each invisible to the single scalar. Existing inference profiling often reports either end-to-end latency or a single utilization axis, with no methodology to say whether a low number comes from memory stalls, tile shape, CTA wave loss, an occupancy cap, attention-side synchronization, or an instruction-set fragment floor. Our goal is therefore not another aggregate benchmark, but a counter-pinned map from the symptom—a high or low utilization reading—to the mechanism that produced it. The contributions below tie each mechanism to its measurement.

Scope. Single H100 NVL, BF16, TP=1, unmodified cuBLASLt/FA3/vLLM, and synthetic prefix-cache states (near-perfect hit on warm prefill). We do not claim the absolute values transfer unchanged to other H100 SKUs,

tensor-parallel configurations, FP8 paths, or production traces with intermediate prefix-cache hit rates; those limitations are revisited in §6.

Contributions. ① *A reusable eight-metric, counter-validated reporting methodology* that replaces the single SM-utilization number with eight complementary views (§5), each — together with its derived quantities — pinned to an NCU counter or explicit formula with verified semantics (Tab. 2). ② *A per-role, per-regime characterization* of the vLLM + FA3 + cuBLASLt serving stack, across cold prefill, warm prefill, and decode on four production models, with every finding tied to a mechanism: dense-GEMM occupancy, under cuBLASLt’s nvjet tile selection, is co-limited by registers *and* shared memory, so a one-sided register cut cannot deepen the warp pool (§5.3); FA3 and dense decode diverge in stall signature, FA3’s leading stall shifting away from memory dependency toward barrier and wait (§5.4); decode-QKV launch grids lose chip coverage to wave quantization (§5.7); and warm prefill collapses toward decode via the warm-cache tile mechanism (§5.6). ③ *A hardware-grounded ceiling on small-batch decode matmul*: Hopper’s GMMA m64 fragment caps every decode dense-GEMM at 1.6–12.5% fragment fill, so a per-kernel compute-throughput reading overstates useful matmul throughput by $8\text{--}64\times$ — a measurement-attribution gap, not recoverable headroom; the lever is to raise the row dimension M (persistent-decode kernels, cross-request token packing), not to retile (§5.6). ④ *Controlled experiments and an artifact release*: a cuBLASLt tile-space enumeration showing no tile achieves full SM coverage for decode-QKV (§5.7), and a $B=32$ validation arm pinning the compute–memory crossover (§5.10). We plan to make the raw NCU and Nsys reports and the derived CSV datasets publicly available.

2. Background and Motivation

2.1. The H100 microarchitectural budget

Our NVIDIA H100 NVL has 132 streaming multiprocessors (SMs). Each SM contains four sub-partitions (warp schedulers), each with one 4th-generation tensor core (TC) — 528 TCs in total [14]. Peak dense BF16 TC throughput is 835 TFLOP/s (1671 TFLOP/s with 2:4 structural sparsity, which we do not exploit) [15]. The BF16 GMMA instruction (wgmma.mma_async) has a fixed 64-row M -axis fragment [14], an instruction-set property we analyze as a small-batch decode ceiling in §5.6. Peak HBM3 bandwidth is 3.9 TB/s [15]. The L2 cache is 60 MB unified across SMs. The per-SM resource budget is 65 536 32-bit registers, 228 KB shared memory (opt-in max), 64 resident warps (architectural ceiling), 32 resident thread blocks, and 4 Tensor Cores. Under the roofline model [23], the arithmetic-intensity ridge is $I^* = 835/3.9 \approx 214$ FLOP/byte in BF16. Kernels below I^* are HBM-bandwidth-bound; kernels above I^* are compute-bound.

2.2. Kernel families in an LLM inference layer

A standard transformer layer in vLLM dispatches the following dense kernel roles per call (Tab. 1): a query–key–value

TABLE 1: Per-layer kernel roles tracked in this study. Dense models use the top five rows; MoE models replace Gate/Up+Down with MOE.

Role	What it captures
QKV	Combined Q/K/V projection (dense GEMM).
O_PROJ	Attention-output projection (dense GEMM).
GATE_UP	FFN Gate/Up projection (dense, fused for SiLU).
DOWN_PROJ	FFN Down projection (dense GEMM).
FLASH_ATTN	FA3 persistent main kernel; the combine kernel is reported under FLASH_COMBINE.
MOE	MoE-only. Fused grouped-GEMM (fused_moe_kernel) plus the small moe_align/moe_sum/scatter-gather kernels that surround it; carries the FFN compute for MoE models. The per-layer router-gate GEMM (hidden $\rightarrow E$ experts) is tracked as a separate ROUTER row (counted in Tab. 4; bucketed into the non-FFN “dense” column of Tab. 6); the topkGatingSoftmax selection kernel is profiled in the CSV cache but not reported.

(QKV) projection, an output projection (o_proj), a Gate/Up projection, a Down projection, and FA3 for attention. The dense BF16 general matrix multiplication (GEMM) kernels are dispatched by cuBLASLt, built on CUTLASS [17], and typically come from the nvjet_hsh_* family on Hopper.

FA3 [22] launches as a *persistent* kernel (FlashAttnFwdSm90, grid $132 \times 1 \times 1$): it launches one block per SM and loops over attention tiles internally, so chip coverage remains full regardless of problem size (§5.7). It is followed by a smaller combine kernel (FlashAttnFwdCombine, grid $5\text{--}80$ cooperative thread arrays (CTAs)).

On the sparse MoE Qwen3-30B-A3B, the feed-forward network (FFN) role is replaced by a fused grouped-GEMM (fused_moe_kernel) that processes the top-8 of 128 experts in a single launch, preceded by a tiny ($\sim 6\ \mu\text{s}$, 2-CTA) router-gate projection. Two auxiliary kernels are tracked but not analyzed individually — CACHE_W (reshape_and_cache_flash, $<2\%$ of decode) and SILU_MUL (FFN SiLU); hence, 7 dense (8 MoE) rows per cell in Tab. 4.

2.3. Three operational regimes

Three regimes exercise the H100 differently. ① **Cold prefill**: every prompt is processed end-to-end with no prior key-value (KV) cache reuse; dense GEMMs see row dimension $M \approx S \cdot B$ and arithmetic intensity is high. ② **Warm prefill**: only a short suffix is re-evaluated after a cached prefix is reused, collapsing M from $\approx S \cdot B$ toward $\approx B$; representative of multi-turn chat. ③ **Decode**: one query token per request per step against a growing KV cache ($M \approx B$); the dominant production regime; HBM-bandwidth-bound.

How the regimes are realised. vLLM’s *prefix cache* [8, 26] reuses already-computed KV blocks across requests (or warmup iterations) sharing a prefix; this flag alone controls the two prefill regimes. A **cold-prefill** cell runs with enable_prefix_caching=false (0% hit by construction), so dense GEMMs see the full $M \approx S \cdot B$ row dimension. A **warm-prefill** cell enables the cache after an identical-prompt warmup; with vLLM’s 16-token cache blocks, the profiled call re-evaluates at most one trailing 16-token block, a deterministic $\geq 99.2\%$ hit at $S=2048$ rising to $\geq 99.9\%$ at

$S=16384$ (decode-sized M). The two prefill regimes therefore bracket the cache-reuse axis at its endpoints; the near-perfect-hit warm case is the small-payload limit, distinct from a realistic intermediate-hit multi-turn workload (§6).

Decode is profiled at `max_tokens=2` with the leading prefill iteration discarded *post hoc*. In every regime, at least three warmup iterations precede the profiled one and CUDA graphs are captured ahead of time.

2.4. Occupancy has two denominators

Achieved occupancy can be normalized against two denominators that often differ by about $5\times$; this is one of the most common and consequential confounds in interpreting GPU utilization. ① **Architectural limit**: 64 warps/SM, a hardware constant independent of the kernel. ② **Kernel resource cap** (NCU’s “Theoretical Active Warps per SM”): the maximum warps the kernel’s resources permit, defined as the minimum over register file, shared memory (SMEM), max blocks, and named barriers; typically 8–12 for the kernels in our sweep, far below 64.

FA3 main is the cleanest illustration: it reaches 7.6 warps/SM, which is 12% of the architectural limit yet 95% of its 8-warp resource cap, i.e. near saturation. Both readings are correct but answer different questions; keeping the normalization explicit resolves much practitioner confusion.

3. Methodology

3.1. Profiling pipeline

All measurements use NVIDIA Nsight Compute 2024.3 (ncu) [16]. We invoke `ncu` per kernel with the `MemoryWorkloadAnalysis`, `LaunchStats`, `SpeedOfLight`, `WarpStateStats`, and `Occupancy` sections enabled. Per-layer, per-role kernel instances (32, 40, 64, and 48 layers for Llama-3-8B, Qwen3-14B, Qwen3-32B, and Qwen3-30B-A3B respectively) are aggregated using NCU’s per-kernel CSV export.

Three metric families (per-stall-reason counters, raw tensor-pipe counters, and the per-call FA3 main-vs.-combine split) are renamed or blended in the upstream CSV, so we re-read the underlying `.ncu-rep` files into three caches: `h100_role_stall_cache.csv` (per-(model, role, regime, S , B) stall counters), `h100_role_tc_cache.csv` (tensor-core counters from `sm_pipe_tensor*`), and `h100_fa_split_cache.csv` (FA3 main and combine separated). NCU reports metrics per kernel launch. For utilization metrics we average matched calls across a model’s layers; for time decomposition we sum across roles within a token. Both views are required: a single per-token aggregate can hide kernel-launch overhead.

3.2. Metrics and their denominators

Tab. 2 defines each metric via its NCU counter or explicit formula. A recurring confound is that nominally similar “utilization” quantities use *different denominators*: SM utilization is elapsed-cycle (not per-active-SM); the two occupancy readings normalize by the 64-warp architectural limit vs. the kernel’s own resource cap; and compute SOL is reported both per active SM and device-wide ($\times$ SMs reached/132,

the Model FLOPs Utilization (MFU)-like form [2]), the two coinciding only when a launch reaches all 132 SMs.

3.3. Working-set model

For the L2-fit analysis, we estimate the per-kernel-call footprint (BF16 bytes) as $\text{footprint} = W_{\text{weights}} + A_{\text{act}} + K_{\text{kv}}$, where W_{weights} is weight bytes read, A_{act} is activation bytes read/written, and K_{kv} is KV-cache bytes read/written. Notation: H hidden size; n_{attn} attention heads; n_{kv} KV heads; d_{head} head dim; $H_q = n_{\text{attn}}d_{\text{head}}$; $H_{\text{kv}} = n_{\text{kv}}d_{\text{head}}$; F FFN intermediate; S context length; B batch size; $T = S \cdot B$ (prefill) or $T = B$ (decode); k experts/token.

4. Workloads, Systems, and Measurement

Hardware and software. We run vLLM (BF16, TP=1) on an H100 NVL with FA3 enabled, the cuBLASLt Hopper GMMMA path, and CUDA Graphs enabled. Graphs stay on for the whole run, including the end-to-end timings we report; the only exception is the NCU-profiled iteration itself, which the profiler requires to be uncaptured so that per-kernel sections remain valid. No wall-clock claim in this paper is taken from a profiled iteration. We run three warmup iterations before each profiled iteration. Model architectural dimensions are taken from public HuggingFace config. json files and listed in Tab. 3.

Configuration sweep. For each model we sweep (S , B) corners; coverage is intentionally asymmetric across regimes. Cold prefill is swept densely on all four models. Warm prefill is profiled at one small corner and two large memory-limit corners per model: because the warm-cache state is shape-insensitive (§2.3), a few corners suffice to bracket it. Decode is swept across $B \in \{1, 2, 4, 8\}$ to expose compute-memory crossover trends; Qwen3-32B decode is the one KV-memory-capped case (256 KB/token forces no $B=8$, and long S admits only $B \leq 4$).

Tab. 4 enumerates the main profiled grid. A *cell* is one profiled (model, regime, S , B) corner; a *row* is one per-role NCU line within a cell, i.e. a (model, regime, S , B , role) tuple, after per-layer aggregation (7 roles for dense models; 8 for Qwen3-30B-A3B, which adds router and fused_moe_kernel and has no separate down-proj role). Warm prefill may additionally expose a `FLASH_COMBINE` role (FA3 combine; otherwise aggregated into `FLASH_ATTN`), so warm-prefill rows are not always a strict cells $\times$ roles product. The $B=32$ validation arm (§5.10) adds four decode cells, one per model ($B=4$ for the KV-memory-capped Qwen3-32B).

5. The Eight-Metric Characterization

The eight metrics are described in the order we recommend for diagnosing an inference-utilization issue.

5.1. SM time-utilization

Fig. 1 reports raw `sm_busy_pct` as a heatmap; the ranges below span all four models and all swept (S , B) corners.

Cold prefill. Dense GEMMs saturate the issue pipeline (73–97%, mean 92%; peak 97.4% on Llama-3-8B `DOWN_PROJ` at $S=8190$, $B=2$). Qwen3-30B-A3B’s FFN (reported under `MOE`) sits at 30–32%. FA3 spans 27–73%, rising with S .

TABLE 2: The eight primary metrics (§5) and their derived quantities. Some metrics occupy two rows where distinct denominators or normalizations apply.

Metric	NCU counter / formula	Interpretation
SM utilization	<code>sm__throughput.avg.pct_of_peak_sustained_elapsed</code> (NCU <i>Compute (SM) Throughput</i> section metric)	Fraction of peak-sustained SM-pipe throughput over elapsed cycles (max over SM sub-pipelines, ideal-SMSP load balance); a time-utilization proxy, <i>not</i> an occupancy, <code>sm__cycles_active</code> , or “ ≥ 1 warp active” counter.
Compute SOL (per active SM)	<code>sm__throughput.avg.pct_of_peak_sustained_elapsed</code> (the same counter as “SM utilization” above)	Peak-sustained SM-pipe throughput read on the SMs the kernel reaches; a duty-cycle proxy for matmul throughput on dense-GEMM kernels.
Compute SOL (device-wide)	<code>compute_sol_pct</code> · SMs_reached/132	The same proxy rescaled by spatial coverage and credited against all 132 SMs; a per-kernel analog of model-level MFU.
SMs used	$\min(N_{CTA}, 132)$ from launch grid	Spatial chip coverage.
Achieved warps/SM	NCU <i>Achieved Active Warps Per SM</i> = <code>sm__warps_active.avg.per_cycle_active</code> (over active SMs; achieved occupancy = <code>sm__warps_active.avg.pct_of_peak_sustained_active</code>)	Resident warp depth.
Occupancy vs. architectural limit	warps/SM $\div$ 64	Saturation vs. the SM’s hardware ceiling (64 warps).
Occupancy vs. kernel resource cap	warps/SM $\div$ (NCU “Theoretical Active Warps per SM”)	Saturation vs. the most warps this kernel’s registers/SMEM allow.
Stall budget	<code>smsp__average_warps_issue_stalled_⟨r⟩_per_issue_active</code> .ratio per reason r , normalized by <code>warp_cycles_per_issued</code>	Why resident warps fail to issue (per-reason share of the per-issued stall budget).
L2 fit ratio ρ	footprint / 60 MB	Per-call working set vs. H100 L2.
Wave efficiency	$N_{CTA} / ([N_{CTA}/132] \cdot 132)$	Launch-grid tiling efficiency.
GMMMA M -fragment fill	$\eta_M = M_{\text{actual}} / ([M_{\text{actual}}/64] \cdot 64)$; the m64 floor is the BF16 <code>wgmma.mma_async</code> architectural minimum [14]	Per-instruction M -axis tile fill; capped above by the WGMMMA hardware fragment (§5.6).
Analytical launch overhead	$\frac{N_{\text{kern}} \tau_{\text{launch}}}{N_{\text{kern}} \tau_{\text{launch}} + \sum_k t_k}$, $\tau_{\text{launch}} = 5 \mu\text{s}$	Share of iteration spent on CPU→GPU dispatch (lower bound).
Kernel-selection diversity	$ \{\text{kernel_name}(s, b)\} $ across the sweep	Number of distinct kernels the autotuner picks; a stability surrogate.

TABLE 3: Model architectures from HuggingFace `config.json` [12, 19]. Qwen3-30B-A3B’s F is per-expert.

Model	H	n_{attn}	n_{kv}	d_{head}	F	L	Type
Meta-Llama-3-8B	4096	32	8	128	14336	32	Dense
Qwen3-14B	5120	40	8	128	17408	40	Dense
Qwen3-32B	5120	64	8	128	25600	64	Dense
Qwen3-30B-A3B	2048	32	4	128	768 (per-expert)	48	Sparse MoE (128 experts, top-8)

Warm prefill. Utilization drops because cuBLASLt’s warm-cache selects smaller tiles: dense GEMMs collapse to 8–11% at ($S=2048, B=1$) and recover unevenly at the larger corner (14–73%, mean 33%). Qwen3-32B’s (16384, 4) warm cell returns to GEMM-like utilization while the others stay decode-like; FA3 is 7–49%; MoE is 26–33%.

Decode. Decode is lowest and relatively uniform: dense 6.5–12.7% (mean 9.4%), FA3 3.4–23.9% (growing with KV working set), and MoE 16–23%. Thus, “decode under-utilizes the GPU” is correct but coarse: small-payload warm prefill is already decode-like, and the cold → warm drop is itself roughly $4\times$ on the dense-GEMM mean. The next seven metrics isolate the mechanisms.

5.2. SM coverage and warp-pool occupancy

Fig. 2 reframes utilization along two additional axes. **SM coverage.** Across the decode sweep, dense projections launch on 80–132 of 132 SMs. QKV grid sizes are model-specific (Llama-3-8B 96 CTAs; Qwen3-14B 132, full; Qwen3-

TABLE 4: Configuration-sweep coverage.

Regime	Model / swept ($S \times B$) corners	Cells	Rows
<i>Cold prefill</i> (prefix caching off)			
	Llama-3-8B $S\{1024..8190\} \times B\{1, 2, 4\} + (8190, 8)$	13	91
	Qwen3-14B $S\{1024..8192\} \times B\{1, 2, 4\} + (16384, 4), (16384, 8)$	14	98
	Qwen3-30B-A3B $S\{1024..8192\} \times B\{1, 2, 4\} + (8192, 8)$	13	104
	Qwen3-32B 12 of $S\{1024..16384\} \times B\{1, 2, 4\}$	12	84
	<i>subtotal</i>	52	377
<i>Warm prefill</i> (prefix caching on, $\geq 99\%$ hit)			
	Llama-3-8B (2048, 1), (8190, 4), (8190, 8)	3	22
	Qwen3-14B (2048, 1), (16384, 4), (16384, 8)	3	22
	Qwen3-30B-A3B (2048, 1), (8192, 4), (8192, 8)	3	25
	Qwen3-32B (2048, 1), (16384, 2), (16384, 4)	3	21
	<i>subtotal</i>	12	90
<i>Decode</i>			
	Llama-3-8B $S\{2048..8190\} \times B\{1, 2, 4, 8\}$	12	84
	Qwen3-14B $S\{2048..8192\} \times B\{1, 2, 4, 8\} + (16384, 8)$	13	91
	Qwen3-30B-A3B $S\{2048..8192\} \times B\{1, 2, 4, 8\}$	12	96
	Qwen3-32B (2048, 1), (16384, 2), (16384, 4)	3	21
	<i>subtotal</i>	40	292
	Total	104	759

Llama-3-8B uses $S=8190$ (2 tokens below its 8192 limit, for decode headroom).

32B and Qwen3-30B-A3B 80; see §5.7 and Fig. 7). `o_PROJ` and `DOWN_PROJ` land at 128; `GATE_UP` lands at 92–132. Thus, limited “SMs reached” is not the dominant decode issue: most SMs do receive work.

Warp-pool depth. Achieved warps/SM is essentially flat across the sweep — ≈ 8.9 – 9.5 for dense BF16 GEMMs (cold

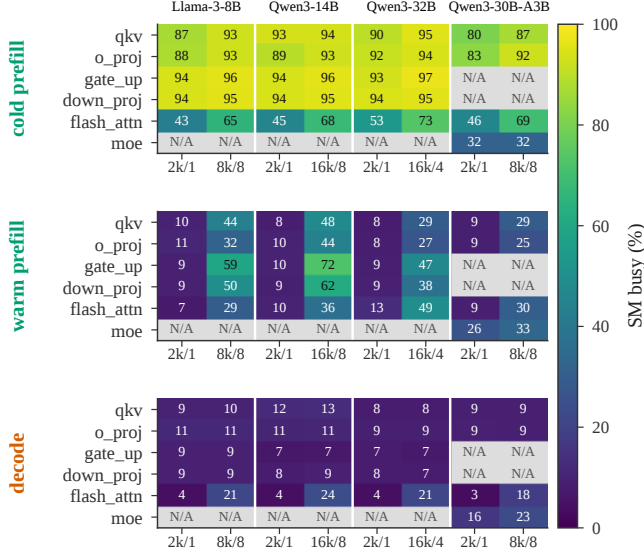


Figure 1: SM utilization heatmaps by model, regime, role, and (S, B) . Rows are kernel roles; columns are swept (S, B) corners; one panel per (model, regime).

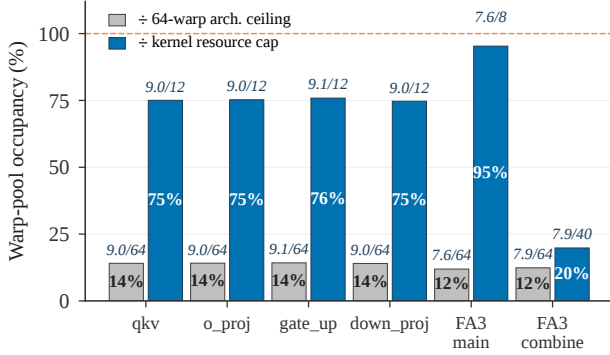


Figure 2: Achieved warps/SM under two denominators on the canonical cell (Llama-3-8B decode, $S=2048, B=1$): normalized by the 64-warps architectural limit and by each kernel’s resource cap η_{cap} (12 dense, 8 FA3 main). Dense decode values stay within 9.0–9.4 warps/SM across the full sweep ($S \in \{2k \dots 16k\}$, $B \in \{1 \dots 8\}$, 8B–32B models): the empty warp pool is not a small-budget artifact, and this cell is representative.

prefill, warm prefill, and decode agree) and 7.6 for FA3 main in decode. Normalized by the 64-warps architectural limit, these correspond to 12–15% (i.e., “the warp pool is mostly empty”). Normalized instead by each kernel’s resource cap (12 for the dense GEMMs, 8 for FA3 main), they are 75–95%, i.e., at or near the kernel’s own ceiling. This $\approx 5\times$ denominator gap from §2.4 holds for every dense decode kernel in our sweep (Fig. 2); it is the rule, not the exception.

5.3. Occupancy cap: registers and shared memory

Section 5.2 shows that the warp-pool cap is set by the *kernel*, not by the architecture. The two resources that most

often bind are the register file and shared memory:

$$R_{\text{eff}} = \lceil R/8 \rceil \cdot 8,$$

$$\eta_{\text{reg}} = \min\left(64, W_{\text{block}} \cdot \left\lfloor \frac{65536}{R_{\text{eff}} \cdot W_{\text{block}} \cdot 32} \right\rfloor\right),$$

$$\eta_{\text{smem}} = \min\left(64, \left\lfloor \frac{228 \cdot 1024}{S_{\text{block}}} \right\rfloor \cdot W_{\text{block}}\right),$$

with binding cap $\min(\eta_{\text{reg}}, \eta_{\text{smem}}, 64)$. Here R is NCU-reported registers per thread and R_{eff} rounds R to Hopper’s 8-register/thread allocation granularity; S_{block} is shared memory per block and W_{block} is warps per block. Per-kernel values are in Tab. 5.

The cuBLASLt rows use the tile that NVIDIA’s nvjet heuristic selects for all four dense projection roles on all four models (Llama-3-8B decode shown; o_proj and down_proj land on the same 12-warps cap as QKV and Gate/Up and are omitted as duplicates). The CUTLASS rows are the three autotuned tiles within a single FP8 kernel (cutlass_3x_gemm_sm90_fp8), covering 15 distinct tile configurations across the four models. The *Bind.* column names the binding resource: “both” when $\eta_{\text{reg}} = \eta_{\text{smem}}$ (co-limited), “smem” when SMEM alone binds.

TABLE 5: Per-resource warps/SM ceilings, grouped by kernel family.

Configuration	Block	R/thr	SMEM	η_{reg}	η_{smem}	Cap	Bind.
<i>cuBLASLt nvjet (BF16) — 4 projection roles</i>							
QKV proj.	384	168	160 KB	12	12	12	both
Gate/Up proj.	384	168	220 KB	12	12	12	both
<i>FA3 (BF16) — decode path</i>							
FA3 main	256	255	147 KB	8	8	8	both
<i>CUTLASS sm90_fp8 (FP8) — 3 autotuned tiles</i>							
Small tile	256	34	227 KB	48	8	8	smem
Medium tile	256	58	198 KB	32	8	8	smem
Large tile	384	168	215 KB	12	12	12	both

FA3 main’s 256-thread block (8 warps, Tab. 5) reflects the use_one_mma_wg decode heuristic in vLLM-FA3: for $\text{seq_len_q} \leq 64$ on sm90, the kernel selects a smaller tile ($\text{kBlockM} = 64$) that allocates 256 registers per MMA thread, driving η_{reg} and η_{smem} both to 8 warps/SM — half the dense-GEMM cap of 12.

The dense BF16 GEMM tile configuration is selected by cuBLASLt’s nvjet heuristic, whose source is proprietary; therefore, the BF16 rows in Tab. 5 are verified via NCU’s runtime-reported occupancy-limiter counters rather than source inspection.

Surprisingly, every dense BF16 kernel we profiled is *co-limited*: η_{reg} and η_{smem} land on the *same* warp count, so neither resource is the sole limiter. Across projection roles, models, and regimes, the cap is uniformly 12 warps/SM with $\eta_{\text{reg}} = \eta_{\text{smem}}$; the underlying cuBLASLt nvjet tiles reduce to a small set of distinct (regs, SMEM, block) configurations, every one balanced. The cap is thus a property of the tiles cuBLASLt’s nvjet heuristic selects for BF16, not of context length, batch size, or Hopper itself: changing (S, B) does not change it.

A separate FP8-E4M3 sweep (the cutlass_3x_gemm_sm90_fp8 path; 4 models, prefill and decode; 11.3k kernel launches) makes this concrete.

The three dominant tile configurations (Tab. 5, lower rows) fall into two groups: (i) two SMEM-only-bound small/medium configurations (34–58 regs/thread, 4–6× register headroom) covering $\approx 90\%$ of instances, and (ii) one register/SMEM co-limited large configuration (168 regs/thread) covering the remaining $\approx 10\%$. The first group caps the kernel at 8 warps/SM through shared memory alone; the second recovers the BF16-style 12/12 balance. The lever is therefore tile- and precision-specific: a one-sided SMEM relaxation deepens the pool only for the SMEM-bound FP8 configs (8→16 warps/SM), never for the co-limited BF16 GEMMs. (FP8 is otherwise out of scope; CUTLASS hand-tuned kernels and decode-attention paths outside cuBLASLt remain untested.)

5.4. Stall-reason decomposition

Given the warps a kernel keeps resident, the next question is why they fail to issue every cycle. We use NCU’s per-issue stall breakdown of `warp_cycles_per_issued`, which we call the *stall budget*: average warp stall cycles per issued instruction, grouped by stall reason.

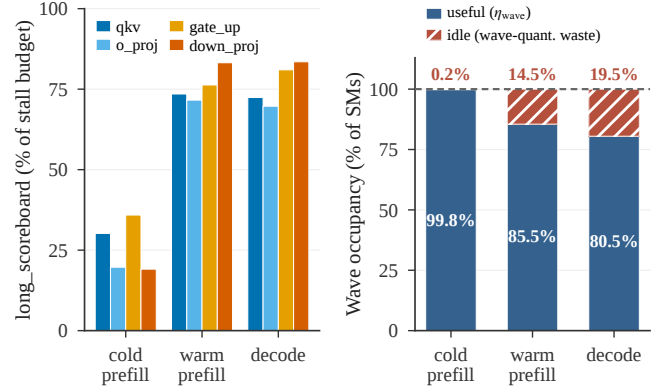
NCU reports seventeen stall reasons; for the dense BF16 GEMMs in our sweep, two dominate. `long_scoreboard` indicates a warp waiting on a long-latency memory load (bandwidth-bound), while `math_pipe_throttle` indicates that the compute pipeline is the bottleneck (compute-bound). Fig. 3a shows the `long_scoreboard` share by regime. The companion panel (Fig. 3b) reports the wave-quantization metric from §5.7; we show them together because they move in *opposite* directions across regimes.

The pattern is clean. Decode and warm prefill are dominated by memory-dependency stalls: `long_scoreboard` accounts for 70–84% of the per-cycle stall budget, consistent with bandwidth-bound execution and substantial headroom in the issue pipeline. Cold prefill drops to 19–36%: the larger cuBLASLt tiles amortize the same loads across more compute, and the remaining stall budget shifts to `gmma`, `barrier`, and `mio_throttle`.

`math_pipe_throttle` does not appear in the top-3 stall reasons for any of the 360 dense-GEMM rows (182 cold-prefill, 42 warm-prefill, 136 decode; one per (model,role, S , B) tuple, all with complete stall counters; FA3/MoE/aux rows excluded). Accordingly, “compute-saturated” is not the right description even for the busiest cold-prefill cells, which are issue-pipeline limited via `gmma` barriers, not pipeline-throttled.

FA3 diverges from this picture as its dominant stalls are `stall_wait/barrier`, not `long_scoreboard`. §5.10 quantifies this contrast and its memory-fix implication.

Concretely, Llama-3-8B QKV at ($S=2048, B=1$) is `long_scoreboard`-dominated and effectively identical in decode and warm prefill — 72.7% and 72.8% of `warp_cycles_per_issued`, respectively. In cold prefill at ($S=8190, B=4$) the same kernel runs the longer cuBLASLt tile at 20.85 cycles per issued instruction, of which only 23.9% are `long_scoreboard`; the largest contributors are `barrier` (27.6%) and `gmma` (23.3%), the remainder `mio_throttle`, `wait`, and the issue slot. The issue pipeline



(a) Memory-dependency stalls. (b) Launch-grid wave quant.

Figure 3: Two regime-resolved metrics for dense BF16 GEMMs, cell-equal-weight mean over the H100 NCU sweep. (a) `long_scoreboard` share of the per-cycle stall budget. (b) Wave efficiency η_{wave} (solid) and wave-quantization waste ($100 - \eta_{\text{wave}}$, hatched: SMs idle in the trailing wave), stacked to 100%.

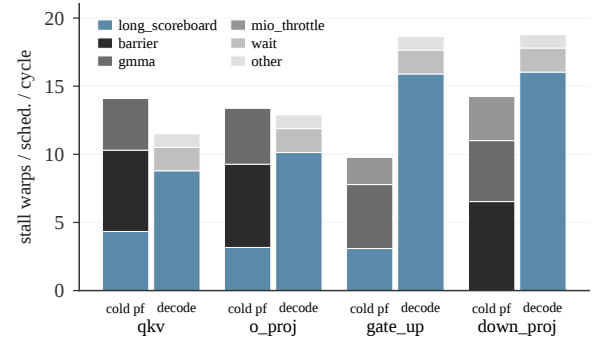


Figure 4: Stall-reason breakdown of `warp_cycles_per_issued` for Llama-3-8B dense BF16 GEMMs (cold prefill (P) vs. decode (D)) at ($S=2048, B=1$). Cold prefill is `barrier/gmma`-dominated, while decode is dominated by `long_scoreboard`. *other* aggregates NCU selected (issue) cycles and remaining minor stalls.

saturates on the GMMA pipeline and inter-warp synchronization, **not** memory. Fig. 4 plots the full breakdown; the per-reason values are concrete fix-it targets for code-generation teams.

5.5. Matmul throughput against the whole array

The per-active-SM and warp-pool metrics quantify how hard the SMs a kernel *occupies* are working, but **not** how much of the H100’s aggregate matmul capacity is exercised over the kernel’s wall-clock. To capture whole-device matmul throughput, we take NCU’s `compute_sol_pct` (the *Compute (SM) Throughput* SOL metric on the SMs the kernel reaches, §3) and scale it by spatial coverage to obtain *device-wide compute SOL*, a per-kernel analog of model-level MFU:

$$\text{device-wide compute SOL} = \text{compute_sol_pct} \cdot \frac{\text{SMs_reached}}{132}.$$

This scaling accounts for spatial *reach* only; it does *not* discount multi-wave tail loss (the wave-efficiency metric of §5.7), so for multi-wave launches it is an upper bound on true elapsed whole-device duty. We also report “effective active SM compute throughput” ($528 \times$ device-wide compute SOL) as a wall-clock-duty upper bound, not a literal Tensor-Core count.

The cross-counter triangulation. The device-wide compute SOL is corroborated by two *independent* NCU counter families read on the *same kernel call*: the DRAM-throughput SOL (`gpu_dram_throughput.avg.pct_of_peak_sustained_elapsed`) and the stall budget (§5.4). On the dense decode GEMM mean the device-wide compute SOL is $\sim 7.9\%$ while the DRAM SOL is $\sim 64\%$ (both elapsed-cycle denominator); the $\sim 8\times$ asymmetry falsifies “the kernel is idle” without invoking stall classifiers. The stall-budget view (§5.4, long_scoreboard 70–84%) is the second corroborating counter family: it is sourced from the per-issue warp-stall counters, independent of `sm_throughput` and of DRAM throughput.

5.6. The GMMMA fragment-fill ceiling on decode matmul

Independent of stalls and spatial coverage, a *kernel-internal* ceiling caps useful matmul on small- M shapes. Hopper’s BF16 GMMMA instruction issues at a fixed $m64 \times n\{8, \dots, 256\} \times k16$ fragment [14]; any GEMM with $M_{\text{actual}} < 64$ still issues full $m64$ instructions and pads the unused rows with zeros. We define the M -axis fragment fill $\eta_M = M_{\text{actual}} / (\lceil M_{\text{actual}} / 64 \rceil \cdot 64)$. Across the 360-row dense-GEMM grid (the dense-role subset of the Tab. 4 rows; 4 models $\times$ 4 roles $\times S \times B$): $\eta_M = 100\%$ in all 182 cold-prefill rows ($M = S \cdot B \gg 64$); $\eta_M = 25 - 100\%$ in warm prefill (42 rows; $M \approx 16B$, mean 70.2%); and $\eta_M = 1.56, 3.12, 6.25, 12.5\%$ in decode for $B \in \{1, 2, 4, 8\}$ (136 rows, mean 5.86%), with *no* exceptions across the four models or four dense roles. This is the kernel-internal complement of the spatial wave-quantization of §5.7: wave efficiency charges idle SMs in the trailing wave; η_M charges idle MMA-rows within each warp-group instruction. Both compound against the device matmul-peak budget.

The consequence is a metric-interpretation failure mode in `compute_sol_pct`. On Llama-3-8B decode qkv at ($S=2048, B=1$), `compute_sol_pct` = 9.5% per active SM, reaching 96 of 132 SMs; device-wide compute SOL is $9.5\% \times 96 / 132 = 6.9\%$. But with $\eta_M = 1.56\%$, the useful FLOP throughput against the 132-SM 835 TFLOP/s peak is $9.5\% \times 1.56\% \times 96 / 132 \approx 0.11\%$ (≈ 0.9 TFLOP/s), because the SM-throughput counter charges padded $m64$ MMAs as work (for a dense BF16 GEMM the tensor pipe is the throughput-limiting sub-pipeline) and cannot distinguish them from useful rows. This is a metric-interpretation claim, not a recoverable-headroom claim: the $M=1, N=6144, K=4096$ shape is bandwidth-bound and no cuBLASLt tile selection ($\text{BLOCK}_M \geq 64$ is hardware-bound; §5.7) can change this. The 6.9% reading therefore overstates useful matmul by $\sim 1/\eta_M \approx 64\times$ rather than indicating that $\sim 90\%$ of the SM compute throughput is recoverable through better tiling.

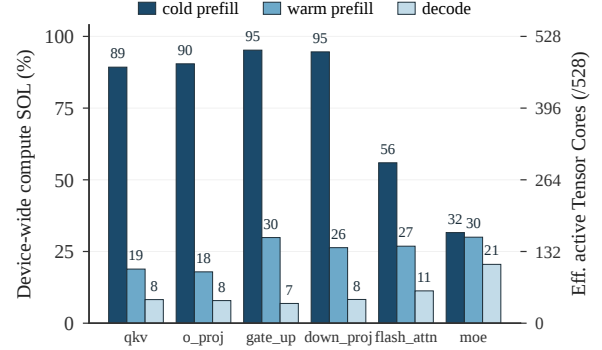


Figure 5: Device-wide compute SOL on H100 by regime (`compute_sol_pct` $\times$ SMs reached/132). Right axis: effective SM compute-throughput equivalents (out of 528), an intuition-only upper bound, not a literal Tensor-Core count. Averaged over the four dense GEMMs, cold prefill reaches 92.0% device-wide SOL but decode only 7.9%, an $11.7\times$ gap.

The lever (§6) is to raise M itself, via persistent-decode kernels or cross-request token packing, not to retile.¹

Small-batch dense decode delivers a low fraction of the chip’s matmul throughput in *absolute* terms, not merely relative to prefill: the device-wide compute SOL means by regime (Fig. 5) put decode at 7.9% of peak against 92.0% for cold prefill and 22.6% for warm prefill, even though decode launch grids reach 96–132 of 132 SMs. The warm figure is low for a different reason, since the cuBLASLt cache has traded the cold tile for an essentially decode-sized one. Per-active-SM compute SOL is the *same* NCU *Compute (SM) Throughput* counter as `sm_busy_pct` — both are `sm_throughput.avg.pct_of_peak_sustained_elapsed`, column-identical across all 759 profiled kernels (max difference 0.16 pp). It differs from the device-wide form only by the SMs_reached/132 factor of §5.5, so it reads higher but tells the same cold/warm/decode story: the dense-GEMM mean is 92/25/9% (Tab. 7), with FLASH_ATTN at 55.9/26.9/11.3% and the MoE-experts kernel at 31.6/30.0/20.5%.

5.7. Wave-quantization waste

The fragment floor of §5.6 wastes rows inside each instruction. The launch grid can waste whole SMs in the same way, one level up. A CUDA grid with N_{CTA} thread blocks (CTAs) executes on 132 SMs in $\lceil N_{\text{CTA}} / 132 \rceil$ waves of up to 132 blocks per wave. The final wave is full only if $132 \mid N_{\text{CTA}}$; otherwise, $132 - (N_{\text{CTA}} \bmod 132)$ SMs sit idle for the entire duration of the trailing wave, yet the kernel is still charged that wave’s wall-clock. Useful array utilization is therefore *quantized* to 132-CTA multiples and rounded up to the next full wave (NVIDIA’s tail/wave-quantization effect).

We quantify this effect with *wave efficiency* η_{wave} , the fraction of per-wave SM slots, summed across all waves, that do useful work: $\eta_{\text{wave}} = N_{\text{CTA}} / (\lceil N_{\text{CTA}} / 132 \rceil \cdot 132)$; and report per-call wasted SM-slot time, $T_{\text{wasted}} = (1 - \eta_{\text{wave}}) \cdot$

1. Throughout, η_M refers to the M -axis only; the GMMMA N -axis ranges 8–256 and is cuBLASLt-selected, so N is not similarly wasted.

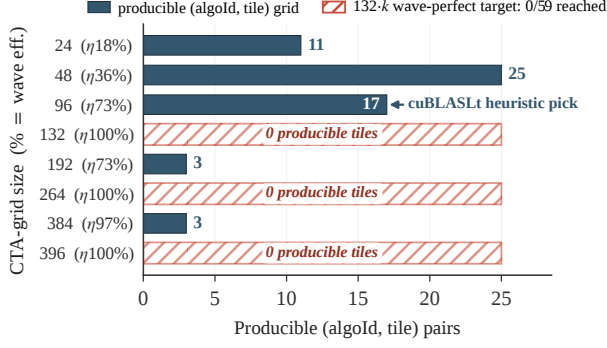


Figure 6: cuBLASLt tile-space enumeration for Llama-3-8B decode-QKV ($M=8$, $N=6144$, $K=4096$, BF16 NT) on H100. Bars count producible (*algold*, *tile*) pairs at each resulting CTA-grid size; η is the corresponding wave efficiency. The three 132- k wave-perfect grids are interleaved as hatched rows: *none* of the 59 producible pairs reaches one, and the heuristic settles on 96 CTAs.

$T_{\text{kernel}} \cdot 132 \text{ SM-}\mu\text{s}$ (idle SM capacity, not elapsed wall-clock). Fig. 3b decomposes the per-regime SM-wave budget into useful η_{wave} and wave-quantization waste ($100 - \eta_{\text{wave}}$): 0.2% in cold prefill, 14.5% in warm prefill, and 19.5% in decode (dense GEMMs, cell-equal-weight mean over the NCU sweep). Worst single calls waste $\sim 21 \text{ K SM-}\mu\text{s}$.

The pathology is sharp but model-specific (Fig. 7). Llama-3-8B QKV launches 96 CTAs ($\eta=72.7\%$, 36 idle SMs, $\sim 21 \text{ K SM-}\mu\text{s}$ wasted per call). Qwen3-32B and Qwen3-30B-A3B launch 80 CTAs ($\eta=60.6\%$, 39% waste). Qwen3-14B happens to fill the chip at 132 CTAs. Other Llama dense projections use $N_{\text{CTA}} \in \{128, 132\}$ ($\eta=97\text{--}100\%$). FA3 is 100% wave-efficient (persistent kernel, internal tile loop). Wave efficiency is the *spatial* complement of the GMMa M -fragment fill η_M (§5.6); the $\text{BLOCK}_M \geq 64$ floor that bounds η_M also bounds the tile-space sweep below (no producible cuBLASLt tile has $\text{BLOCK}_M < 64$).

We then ask whether a 132-multiple tile exists within cuBLASLt’s algorithm space. We swept the full $17 \times 12 = 204$ -entry space of cuBLASLt’s 17 BF16 NT algoIds $\times$ 12 tile shapes for the Llama-3-8B decode-QKV problem ($N=6144$, $K=4096$) at the M where cuBLASLt selects the 96-CTA tile ($M=8$; the heuristic holds the same tile across $M \in \{8, 16, 32, 64, 128\}$). Of the 17 algoIds, 10 expose no usable tile for this shape, leaving 59 (*algoid*, *tile*) pairs. The finding is clearly *negative*: zero of the 59 pairs yields a 132-multiple grid for this shape (Fig. 6), every producible grid falling in $\{24, 48, 96, 192, 384\}$.

The reason is structural: since $132 = 4 \cdot 3 \cdot 11$ contains a factor 11 and QKV has $N=6144 = 2^{11} \cdot 3$, no power-of-2 tile shape can produce a CTA grid size $\lceil M/M_t \rceil \cdot \lceil N/N_t \rceil \in \{132, 264, \dots\}$ for these dimensions. A 132-multiple grid would need either a non-standard tile factor (e.g., a 559–614-wide N -tile, giving exactly 11 N -tiles) or split- K with factor 11, neither present in cuBLASLt’s current nvjet tile set for this shape. The 96-CTA tile cuBLASLt selects is nonetheless defensible on per-CTA efficiency grounds: enumerated alternatives that fill the chip better (e.g., 16×16 at 384 CTAs,

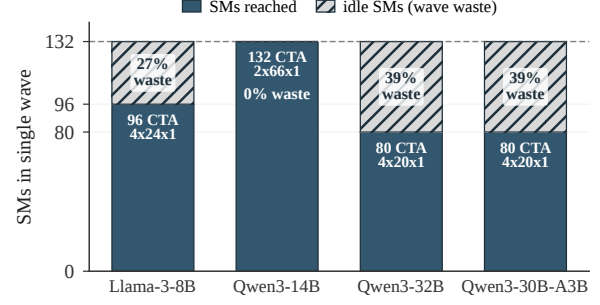


Figure 7: Decode-QKV grid CTA count per model: 132-CTA (Qwen3-14B, 0% waste), 96-CTA (Llama-3-8B, 27%), 80-CTA (Qwen3-32B and -30B-A3B, 39% each). Grid is invariant across all swept $S \in \{2048, \dots, 16384\}$, $B \in \{1, 2, 4, 8\}$.

97% wave efficiency) shrink per-CTA work substantially, and cuBLASLt’s heuristic optimizes throughput rather than wave fill alone.

5.8. Memory hierarchy and L2 working-set fit

Decode is HBM-bandwidth bound (§5.4), but separating kernels that are limited by L2 capacity from those limited by HBM bandwidth is a finer question. We use the L2 fit ratio $\rho = \text{footprint}/60 \text{ MB}$, where $\rho > 1$ indicates that the working set spills L2 across calls.

Decode FFN projections (gate_up, down_proj) spill L2 by 2–4 $\times$ across all dense models, consistent with the low observed L2 hit rates. Decode QKV is dominated by its weight matrix ($\sim 48 \text{ MB}$ on Llama-3-8B), so fits L2 ($\rho \approx 0.80$), whereas the larger Qwen QKV weights spill (1.2–1.7 $\times$).

Across all (kernel, S , B) combinations, 90.0% spill in cold prefill, 92.5% in warm prefill, and 53.8% in decode. The per-call MoE moe row spills less aggressively than dense FFNs at $B=1$: top-8 of 128 experts touches roughly 1/16 the dense-FFN working set, so the per-call footprint is correspondingly smaller. At $B>1$, up to $\min(Bk, E)$ distinct expert weight matrices may be loaded per call, raising the effective MoE weight footprint by up to $\min(B, 16)\times$; thus, the $\rho \approx 1.20$ figure is a lower bound at larger batches (see the working-set caveat in §6).

5.9. Compute-bound crossover batch

Relative to the BF16 roofline ridge $I^* \approx 214 \text{ FLOP/byte}$ (§2), we capture where each role lives with the compute-to-memory SOL ratio $r(B) = \text{compute_sol_pct}(B)/\text{memory_sol_pct}(B)$, and call the smallest B for which $r(B) \geq 1$ – a SOL-balance proxy for the roofline ridge (achieved tensor- vs. memory-pipe duty, **not** an arithmetic-intensity computation) – the *crossover batch* B^* . Within the swept range ($B \leq 8$), no decode dense-GEMM crosses for any model. Per-role r values are uniformly low and tightly clustered for the dense projections (qkv 0.21, o_proj 0.26, gate_up 0.12, down_proj 0.13); flash_attn alone is closer at 0.84. Linear extrapolation puts B^* at ~ 31 , ~ 25 , ~ 49 , ~ 45 , and ~ 8 respectively. Wave quantization (§5.7) therefore dominates well before B^* is reached. Production decode rarely runs $B \geq 25$ per request

TABLE 6: Decode microarchitecture at $B=32$ on H100 with FA3. Columns report per-iteration time shares for the three largest buckets (dense GEMMs / FA3 / MoE experts), SM-busy for dense GEMMs, and per-token decode time (ms/tok). Shares are not a partition and may not sum to 100%.

Model	(S, B)	dense%	FA3%	moe%	SM-busy	ms/tok
<i>B=32 validation arm (B at memory cap):</i>						
Llama-3-8B [§]	(8190, 32)	58.2	32.7	—	51.7	1.52
Qwen3-14B	(16384, 32)	44.0	49.5	—	71.9	2.38
Qwen3-30B-A3B	(8192, 32)	11.4	31.6	36.2	34.4	0.96
Qwen3-32B	(16384, 4)	69.6	18.8	—	23.8	18.4
Qwen3-32B	(2048, 32)	79.2	11.6	—	72.6	0.98
<i>B=1 reference (matched S except where footnoted):</i>						
Llama-3-8B [§]	(8190, 1)	80.5	9.8	—	~ 10	5.50
Qwen3-14B	(16384, 1)	80.8	10.5	—	~ 12	10.10
Qwen3-30B-A3B	(16384, 1) [†]	21.8	15.2	35.7	~ 9	5.32
Qwen3-32B	(16384, 1) [‡]	85.8	7.6	—	~ 8	22.91

[†] $B=1$ ref reported at $S=16K$ (long-context regime); the cross- S comparison to the (8K, 32) cell is sound because dense SM-busy is S -invariant (8.9–9.0% at all swept S , incl. 8K).

[‡]shares at $S=16K$; ms/tok column is at $S=32K$ (the available cell).

[§]meta-llama/Meta-Llama-3-8B has max_position_embeddings=8192; $S=8190$ leaves 2-token decode headroom under that 8192 limit (8190+2=8192), not a measurement choice.

because of the latency service-level objective (SLO), so dense decode GEMMs stay memory-bound.

5.10. Empirical validation at $B=32$

The crossover batches in §5.9 are linear extrapolations beyond the swept $B \leq 8$ range; without a measurement at $B \geq 25$ the prediction is unfalsifiable. We therefore extend the sweep with one $B=32$ validation H100/FA3 NCU pass per model at the largest B the KV-memory budget admits: $B=32$ at $S=8190$ on Llama-3-8B, $S=16384$ on Qwen3-14B, and $S=8192$ on the sparse-MoE Qwen3-30B-A3B; the 62 GB Qwen3-32B caps at $B=4$ at $S=16384$ under the same 0.92 gpu_memory_utilization, and so appears twice in Tab. 6 ($B=4$ at long S , $B=32$ at short S). Every other lever (vLLM 0.16, FA3, BF16, prefix-caching ON, ≥ 3 warmups) is held identical to the headline sweep. Tab. 6 reports before/after comparisons: each $B=32$ validation row (top panel; five rows) against the same model’s $B=1$ reference (bottom panel), at matched context length where the KV budget allows and cross- S otherwise (footnoted).

Crossover and bottleneck shift at $B=32$. On Qwen3-14B, decode dense-GEMM SM-busy rises from $\sim 12\%$ at $B=1$ to **71.9%** at $B=32$; FA3-fwd reaches **49.5%** of decode iteration, overtaking dense GEMMs (44.0%); per-token wall-clock falls from 10.10 to 2.38 ms ($\sim 4.2\times$); long_scoreboard on dense GEMMs drops from 26 to 12 cycles/issue and DRAM busy from 39 to 36% — the measured cell lands near the predicted $B^* \approx 31$ for QKV. Llama-3-8B approaches but does not cross: SM-busy 51.7% ($5\times$ the $B=1$ reference), FA3 32.7%, dense still 58.2%; its smaller hidden $F=14336$ (vs. 17408) places B^* at ~ 49 for GATE_UP, beyond a feasible SLO batch. On Qwen3-30B-A3B the bottleneck shifts: the routed-FFN share of decode time (the fused_moe_kernel% column of Tab. 6: fused_moe_kernel plus its align/sum/scatter aux, router excluded) is essentially flat from 35.7% at $B=1, S=16K$ to 36.2% at $B=32, S=8K$, while FA3-fwd *more than doubles*

from 15.2% to 31.6%. At $B=32$ the model is jointly routed-FFN and FA3 bound rather than routed-FFN dominated, and the two share decode time within 5% points.

Qwen3-32B crosses at short S but is KV-forbidden at long S : at the long-context cell ($B=4, S=16K$) dense SM-busy reaches 23.8% — above the small-batch mean of 8.2% but far from $B^* \approx 49$. At ($B=32, S=2048$) the same kernels reach **72.6%** SM-busy — matching Qwen3-14B’s 71.9% at (16K, 32), i.e. the dense stream crosses *despite* the 32B parameter count — and per-token wall-clock collapses to 0.98 ms, on par with the sparse MoE’s 0.96 ms at (8K, 32) even though the MoE activates only $\sim 3B$ params/token. The crossover is a B effect on the dense stream; the KV footprint (KV bytes/token = $2 \cdot n_{kv} \cdot d_{head} \cdot 2 \cdot L$: 256 KB for Qwen3-32B with $n_{kv}=8, L=64$, vs. 96 KB for the MoE with 4, 48) constrains *when*, not *whether*: $B=32$ needs $S \lesssim 3K$ on the 94 GB part, while long- S (16K) loads are KV-capped to $B \leq 4$ in our sweep (§4). This refines the right-batching implication (§6): shifting the operating-point question from “can model M cross?” to “does (S, B) admit it?”

FA3’s stall signature stays distinct from dense decode at $B=32$: dense GEMMs keep the canonical long_scoreboard (memory-dependency) signature (11.5–26.4 cycles/issue), but FA3-fwd’s top stall is stall_wait (barrier/fixed-latency dependency, 1.8) with long_scoreboard only 1.1–1.2: HBM upgrades move dense GEMMs but not FA3, and the FA3 share growing to $\sim 50\%$ of decode reflects attention compute scaling with B , not a hidden bandwidth bottleneck. This sharpens the FA3-rework implication (§6): $B=32$ decode latency needs kernel-side rework on FA3, distinct from the memory-hierarchy levers that address the dense stream.

Finally, KV-cache write closes the cost composition at $< 2\%$ even at $B=32$: the reshape_and_cache_flash kernel stays at 0.7–1.6% of decode iteration across all four $B=32$ cells (Llama-3-8B 1.19%, Qwen3-14B 0.69%, Qwen3-30B-A3B 1.55%, Qwen3-32B 1.26%). The atomic decomposition used by chunked-prefix / PD-disaggregation cost models [1, 18, 27] — $T_{\text{iter}} \approx T_{\text{dense}} + T_{\text{FA3}}$ for dense models, $T_{\text{iter}} \approx T_{\text{dense}} + T_{\text{FA3}} + T_{\text{moe}}$ for the MoE — therefore needs no separate T_{KV} term: the KV-write residual stays $\leq 2\%$ through $B=32$.

5.11. Kernel-selection stability

Stable kernel selection is a precondition for hand-tuning and persistent-kernel rewrites. We measure *kernel-selection diversity* as the number of distinct kernel names that cuBLASLt selects for each (model, regime, role) across the sweep. In decode, this count is uniformly 1 for dense GEMMs and 2 for FA3; cold-prefix QKV peaks at 7 (Llama-3-8B). Thus, decode underperformance is not caused by unstable kernel selection: cuBLASLt selects kernels stably, and the selected kernels remain *inefficient* — a favorable setting for persistent-kernel rewrites.

5.12. Per-token time decomposition

Kernel-level bottlenecks matter in proportion to their per-token time share. In the main $B \leq 8$ sweep, the FFN is the dominant cost at 58–66% mean across models and regimes (the $B=32$ validation arm shifts toward FA3; §5.10). For

TABLE 7: Cross-metric, cross-regime summary on dense BF16 GEMMs. The metrics split into *code-generation levers* (warps/SM, wave efficiency — largely flat across regimes) and *workload outcomes* (sm_busy_pct, compute SOL, L2 fit, launch overhead, stalls — order-of-magnitude shifts cold→decode); this separates fixes that live in cuBLASLt/compilers/serving from those that require workload restructuring. The launch-overhead row is analytical ($\tau=5\ \mu\text{s}$).

Metric	Cold prefill	Warm prefill	Decode	Cold→decode
SM time-utilization (sm_busy_pct)	73–97%	8–73%	6.5–12.7%	$\sim 10\times$ (mean 92% vs. 9.4%)
Achieved warps/SM	8.9–9.5	8.8–9.5	8.8–9.5	$\approx 1\times$ (flat)
Compute SOL, per active SM (dense GEMM mean)	92%	25%	9%	$\sim 10\times$
Compute SOL, device-wide	92.0% (mean), 97% (peak)	22.6%	7.9%	$\sim 11.7\times$
Wave efficiency (mean)	99.8%	85.5%	80.5%	$\sim 1\times$
GMMA M -fragment fill η_M (dense GEMM mean)	100%	70.2%	5.86% (1.6–12.5%)	$\sim 17\times$
L2 fit ratio ρ (mean dense; $\rho>1$ spills)	$14.7\times$	$16.0\times$	$2.0\times$	$\sim 8\times$ smaller (still spills $2\times$)
Analytical launch overhead (Llama-3-8B)	1.3%	28.5%	29.9%	$\sim 23\times$
long_scoreboard % of stalls	19–36%	72–83%	70–84%	$\sim 3\times$

example, Llama-3-8B decode at ($S=2048, B=1$) takes 5.27 ms per token, of which 63.7% is FFN (gate_up 41%, down_proj 22%), 30.8% attention, 3.2% norm/embed, and 2.2% reduce. FFN is therefore the natural denominator for “X% improvement” claims, including the compute-SOL, wave, and launch-overhead fixes of §6.

6. Discussion

Implications. The findings translate into levers at three layers. *Software and serving*: batch to the compute-memory crossover rather than to “more batch” — $B^*\in[25, 49]$ (§5.9) bounds where dense decode leaves the bandwidth-bound regime, a concrete target for batching, speculative decoding [10], and PD-disaggregation; and schedule decode and warm prefill separately, since warm prefill is closer to decode than to cold prefill (§5.6). Decode’s analytical launch overhead reaches $\sim 30\%$ of the iteration (Tab. 7), so a dispatch-amortizing decode path (graph capture or a persistent kernel) helps.

Libraries and code generation: the largest decode losses are not retiling problems — no cuBLASLt tile yields a 132-multiple grid for decode-QKV (§5.7), warp-pool headroom needs register and SMEM cuts together (§5.3), and the GMMA $m64$ floor caps every decode cell at $\eta_M=B/64$ regardless of tile choice (§5.6). It is worth separating two levers that are easily conflated. A persistent decode kernel removes launch overhead and wave-quantization loss, but it leaves η_M unchanged, because it adds no token rows. Continuous batching does not raise η_M either, since it already sets the decode-GEMM row count to $M=B$ (§2.3) and the latency target holds B below the crossover of §5.9. Rows can only come from speculative, tree, or multi-token decoding, which give $M=kB$ and fill the fragment at $k=8, B=8$.

Architecture: decode stresses memory and host dispatch, not the tensor pipe, raising the open question of whether finer-grained power/clock gating of the tensor datapath is warranted; and decoupling the register file from shared memory would let one resource give occupancy headroom instead of both having to move. Finally, operators should report multiple denominators: a kernel can look healthy on SM-time and warp axes while delivering a small fraction of peak matmul throughput.

Threats to validity. Six caveats qualify our claims. ❶ *FA3 row blending*: the standard flash_attn row blends FA3 main and combine, so reading it as main alone underestimates decode FA3 duty (we re-read the two separately from the .ncu-rep files; §3). ❷ *Linear B^* extrapolation* is validated and refined by the $B=32$ arm (§5.10: Qwen3-14B crosses near $B^*\approx 31$, Llama-3-8B approaches, Qwen3-32B crosses at short S but is KV-capped at long S , Qwen3-30B-A3B shifts to a routed-FFN/FA3 co-bottleneck) — a diagnostic, not a predictor. ❸ *Single-GPU, $TP=1$, single H100 SKU*: TP alters per-GEMM shape and adds all-reduce, and HBM-bound numbers may shift on other H100 SKUs. ❹ *Launch-overhead constant*: the analytical $\tau=5\ \mu\text{s}$ is host-/driver-dependent and captures raw dispatch only, not runtime-path/replay/scheduling cost. ❺ *Working-set model is first-order*: no scratch/tile staging; within ~ 10 – 20% of dram_bytes on cross-checked kernels. ❻ *Warm-prefill cache state*: synthetic warmup yields $\geq 99\%$ prefix-cache hit; realistic multi-turn intermediate hits are uncharacterized.

7. Related Work

GPU characterization and roofline. Prior LLM inference work on A100/H100 [7] measures end-to-end performance. We instead map per-role kernels to NCU counters, validate with calibration, and tie analysis to measurement via a controlled cuBLASLt tile-space enumeration. Hopper micro-benchmarking [13] studies primitives in isolation; we show how an end-to-end serving stack (vLLM/FA3/cuBLASLt) composes them. Roofline [23–25] is the main analytical precedent; we extend it with wave quantization, kernel-selection stability, and a cuBLASLt algorithm-space enumeration. Our counter-level results (the FA3 blending caveat and the register/shared-memory occupancy co-limit) refine the meaning of NCU metrics [16]. FlashDecoding++ [5] also treats the small- M decode GEMM as a first-order problem and builds a flat-GEMM kernel that pads M to 8 instead of 64. The two efforts meet from opposite directions. That work proposes a kernel; we measure what the stock library path actually does and show why the row deficit survives every tile the library can produce (§5.7). Read together, our enumeration explains why a custom kernel is the level at which the problem has to be solved.

MoE, profiling, and benchmarks. MoE work is mostly training-focused, covering routing, expert parallelism, load balancing, grouped-GEMM kernels, and distributed-MoE imbalance [3, 4, 6, 9, 11, 20]. Benchmark suites such as MLPerf [21] report end-to-end latency or throughput; we instead report per-role microarchitectural metrics.

8. Conclusion

SM utilization is a useful number, but it is not a diagnosis. In LLM inference, the same kernel can look busy or idle depending on whether we count elapsed SM activity, tensor-pipe work, launch-grid coverage, occupancy headroom, or useful GMMa fragment fill, and collapsing these views into one scalar hides where performance is actually lost. Decode makes this concrete. Dense decode GEMMs often reach most of the chip, yet at small batch sizes Hopper’s BF16 GMMa m64 fragment floor leaves most rows of every instruction empty, so SM-busy stays high while useful matmul does not. This is an instruction-set ceiling, not a tiling choice, and no cuBLASLt retuning can lift it. FA3 decode loses time to a different cause, barrier-style waits rather than memory stalls. Different bottlenecks need different fixes: small-batch decode is improved less by retuning cuBLASLt tiles than by raising the effective row dimension M , through cross-request token packing or persistent decode kernels. More broadly, LLM inference studies should report counter-pinned, per-role metrics rather than a single device-wide utilization percentage.

Acknowledgments

We would like to thank the anonymous reviewers for their insightful comments and suggestions on this paper. This work has been partially supported by Vinnova (Sweden’s Innovation Agency), the Swedish Research Council (agreement No. 2021-04212), KTH Digital Futures, and the Knut and Alice Wallenberg Foundation (Wallenberg Scholar Grant for Prof. Dejan Kostić).

References

- [1] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, “Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 117–134. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/agrawal>
- [2] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, “Palm: Scaling language modeling with pathways,” *Journal of machine learning research*, vol. 24, no. 240, pp. 1–113, 2023.
- [3] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: scaling to trillion parameter models with simple and efficient sparsity,” *J. Mach. Learn. Res.*, vol. 23, no. 1, Jan. 2022.
- [4] T. Gale, D. Narayanan, C. Young, and M. Zaharia, “Megablocks: Efficient sparse training with mixture-of-experts,” *Proceedings of Machine Learning and Systems*, vol. 5, pp. 288–304, 2023.
- [5] K. Hong, G. Dai, J. Xu, Q. Mao, X. Li, J. Liu, K. Chen, Y. Dong, and Y. Wang, “FlashDecoding++: Faster large language model inference with asynchronization, flat GEMM optimization, and heuristics,” in *Proc. Machine Learning and Systems (MLSys)*, vol. 6, 2024, pp. 148–161.
- [6] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang, R. Salas, J. Jose, P. Ram *et al.*, “Tutel: Adaptive mixture-of-experts at scale,” *Proceedings of Machine Learning and Systems*, vol. 5, pp. 269–287, 2023.
- [7] J. Kundu, W. Guo, A. BanaGoza, U. De Alwis, S. Sengupta, P. Gupta, and A. Mallik, “Performance modeling and workload analysis of distributed large language model training and inference,” in *2024 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2024, pp. 57–67.
- [8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM Symp. on Operating Systems Principles (SOSP)*, 2023.
- [9] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, “GShard: Scaling giant models with conditional computation and automatic sharding,” in *Int. Conf. on Learning Representations (ICLR)*, 2021.
- [10] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proc. Int. Conf. on Machine Learning (ICML)*, 2023.
- [11] M. Lewis, S. Bhosale, T. Dettmers, N. Goyal, and L. Zettlemoyer, “Base layers: Simplifying training of large, sparse models,” in *International Conference on Machine Learning*. PMLR, 2021, pp. 6265–6274.
- [12] Llama Team, AI @ Meta, “The Llama 3 herd of models,” arXiv preprint arXiv:2407.21783, 2024.
- [13] W. Luo, R. Fan, Z. Li, D. Du, Q. Wang, and X. Chu, “Benchmarking and dissecting the NVIDIA Hopper GPU architecture,” in *Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS)*, 2024.
- [14] NVIDIA Corporation, “NVIDIA H100 Tensor Core GPU Architecture,” Whitepaper, 2022, accessed: 2026-05-21. [Online]. Available: <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>
- [15] NVIDIA Corporation, “NVIDIA H100 NVL GPU product brief, PB-11773-001_v01,” https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/h100/PB-11773-001_v01.pdf, 2024.
- [16] NVIDIA Corporation, “NVIDIA Nsight Compute documentation (version 2024.3),” <https://docs.nvidia.com/nsight-compute/>, 2024, accessed: 2026-05-21.
- [17] NVIDIA Corporation, “NVIDIA CUTLASS: CUDA templates for linear algebra subroutines and solvers,” <https://github.com/NVIDIA/cutlass>, 2026, accessed: May 21, 2026.
- [18] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative llm inference using phase splitting,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 118–132.
- [19] Qwen Team, “Qwen3 technical report,” arXiv preprint arXiv:2505.09388, 2025.
- [20] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, “DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation ai scale,” in *International conference on machine learning*. PMLR, 2022, pp. 18 332–18 346.
- [21] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, “MLPerf inference benchmark,” in *Proc. 47th Int. Symp. on Computer Architecture (ISCA)*, 2020.

- [22] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, ser. NIPS '24. Red Hook, NY, USA: Curran Associates Inc., 2024.
- [23] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [24] C. Yang, T. Kurth, and S. Williams, "Hierarchical Roofline analysis for GPUs: Accelerating performance optimization for the NERSC-9 Perlmutter system," *Concurrency and Computation: Practice and Experience*, vol. 32, no. 20, 2020.
- [25] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, Z. Zhou, C. Xue, B. Wu, Z. Li, Q. Gu, Y. J. Lee, Y. Yan, B. Chen, G. Sun, and K. Keutzer, "LLM inference unveiled: Survey and roofline-model insights," *arXiv preprint arXiv:2402.16363*, 2024.
- [26] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [27] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. 18th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2024.